

Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

by

Ratul Sikder

Submitted to the Institute of Information Technology (IIT)
in partial fulfillment of the degree requirements

at the

UNIVERSITY OF DHAKA

August 2026 — Condensed Edition

© University of Dhaka 2026. All rights reserved.

The author hereby grants the Institute of Information Technology (IIT), University of Dhaka, permission to reproduce and distribute copies of this project report, in whole or in part, for academic purposes.

Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

by

Ratul Sikder

Submitted to the Institute of Information Technology (IIT) on 11 August 2026,
in partial fulfillment of the degree requirements

This is to declare that this project is the author's original work. No part of it has been submitted elsewhere, in whole or in part, for the award of any other degree or diploma, and the plagiarism policy stated by the supervisor has been maintained.

Supervisor's Name: _____

Signature of the Supervisor: _____

Date: _____

Abstract

Modern software engineering teams spend an estimated 40 to 60 percent of their time on maintenance, debugging, and manual incident response, rather than on building new capability. A number of AI-assisted developer tools already exist, such as code completion assistants, automated test generators, and deployment dashboards, but each of these addresses only an individual task in isolation. None of them model the collaborative structure of a real engineering team, and none of them close the loop between detecting a production failure and actually repairing it. **Nexis** has been designed to address both of these gaps together. The system models a software engineering organisation as nine role-specialised autonomous agents, arranged across two cooperating layers: a five-agent Execution Team (Architect, Backend, Quality Assurance (QA), DevOps, and Data Engineer) and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, and Validator). These agents are durably orchestrated using Temporal, and every automated repair is gated by a severity-routed, human-in-the-loop Approval Gate that supports Approve, Reject, and Modify (Reinforcement Learning from Human Feedback (RLHF)) decisions. This report documents the complete, working system, which includes a multi-tenant control plane built with Go (Golang), PostgreSQL with Row-Level Security (RLS), Neo4j, and Redis; a Docker shadow-validation sandbox; a real GitHub-App-mediated GitOps deployment path; and a from-scratch Docker preview-deploy engine. It also includes a Next.js console that spans authentication, Role-Based Access Control (RBAC), incident observability, billing, and multi-provider integrations. Two live end-to-end recovery runs against a local Large Language Model (LLM) have been reported verbatim, alongside a live and verified proof of the preview-deploy engine. What has been deliberately left out of scope, namely a fully credentialed production deployment to the registered `nexis.sh` domain and the human-subject NASA Task Load Index (NASA-TLX) study, has been stated plainly rather than left implied.

Keywords: Multi-agent systems, autonomous software engineering, self-healing systems, closed-loop fault recovery, root-cause analysis, LLM-guided program repair, DevOps automation, Temporal workflow orchestration, human-in-the-loop AI.

This is the condensed edition of the full project report. The complete 107-page edition with exhaustive feature-by-feature screenshots is available as a companion document.

Table of Contents

Chapter 1: Introduction.....	5
1.1 Background.....	5
1.2 Motivation.....	5
1.3 Problem Statement.....	5
1.4 Project Objectives.....	5
1.5 Scope of the Project.....	6
1.6 Significance of the Project.....	6
Chapter 2: Literature Review.....	7
2.1 Overview.....	7
2.2 Research Gap.....	8
2.3 Conclusion.....	8
Chapter 3: System Analysis and Requirements.....	9
3.1 Requirement Analysis.....	9
3.2 Functional Requirements.....	9
3.3 Non-Functional Requirements.....	10
3.4 System Feasibility.....	11
Chapter 4: System Design and Architecture.....	12
4.1 System Context.....	12
4.2 Component Architecture.....	12
4.3 Data and Storage Architecture.....	13
4.4 Self-Healing Loop Pipeline.....	14
4.5 Deployment and Infrastructure.....	14
4.6 Use Case Diagrams.....	15
4.7 Data Flow Diagrams.....	16
4.8 Self-Healing Loop Sequence Diagram.....	18
Chapter 5: Implementation.....	20
5.1 Tools and Technology.....	20
5.2 Project View.....	20
5.3 Database Schema.....	29
Chapter 6: Testing and Results.....	31
6.1 Static Verification.....	31
6.2 Live End-to-End Recovery Runs.....	31
6.3 Live Deploy-Engine Verification.....	32
6.4 Defects Found and Fixed During This Engineering Cycle.....	32
6.5 Testing Summary.....	32
Chapter 7: Conclusion and Future Work.....	33
7.1 Overview.....	33
7.2 Limitations of the Project.....	33
7.3 Future Work.....	34
7.4 Conclusion.....	34
References.....	35

Chapter 1: Introduction

This chapter lays out the background, motivation, problem statement, objectives, scope and significance of **Nexis**.

1.1 Background

Software rarely fails when it is written; it fails later, in production, under load, against data and dependencies nobody anticipated. Industry experience cited in the project proposal puts 40–60% of engineering time into maintenance, debugging and incident response rather than new capability, meaning roughly one engineer repairing for every engineer building.

Assistants such as GitHub Copilot [1] now write a meaningful share of new code, yet each one accelerates a single task for a single human. The production incident lifecycle (detect a fault, diagnose the root cause, fix, validate, obtain approval, deploy, verify) remains a chain of manual hand-offs, with the engineer acting as transport between every stage.

Nexis closes that chain. It models an engineering organisation as nine role-specialised autonomous agents in two layers, a five-agent Execution Team and a four-agent Self-Healing Loop, orchestrated by the Temporal workflow engine [2], with human approval reduced to a single severity-routed decision.

1.2 Motivation

Several pain points motivate this work:

- **No tool closes the loop.** Sentry [14], Datadog [16] and PagerDuty [15] detect failures and page a human; Copilot [1] writes code once it is asked to; nothing yet connects detection to autonomous repair.
- **The engineer is the bottleneck for every incident**, even where the fault class recurs in an almost identical form: reading logs, localising, patching, testing, reviewing, deploying.
- **Root-cause analysis goes unaided.** No mainstream tooling walks the code’s dependency graph or ranks candidate causes with stated evidence.
- **Repair without validation is dangerous, so nobody attempts it.** Without a sandboxed path for a machine-generated patch to prove itself, organisations reasonably refuse to trust it, and the loop stays open.
- **Approval is all-or-nothing:** gates are either absent, which makes automation risky, or heavyweight, requiring full review even for a one-line fix; nothing routes human involvement by measured severity.

1.3 Problem Statement

Taken together, the limitations of existing approaches are structural: isolated tools, no closed loop, a human bottleneck on every decision, no causal root-cause tooling (Neo4j [3] graph traversal, DoWhy [4] causal reasoning), and no safe autonomous deploy path with sandboxed validation and post-deploy rollback.

Hence, the problem may be stated as follows: *how can the full production-fault lifecycle be closed autonomously by cooperating role-specialised agents, while keeping a human decision in the loop exactly where, and only where, the severity of the change warrants it?*

1.4 Project Objectives

Nexis set out to build the following, and delivered each as described below:

1. **A nine-agent, two-layer architecture.** A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) producing plans, minimal-diff patches, property-based tests, manifests and migrations; and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detecting faults, walking the Neo4j [3] dependency

graph towards root causes, and shadow-executing every patch in a locked-down Docker [10] sandbox, the whole pipeline orchestrated as a durable, replayable Temporal workflow [2].

2. **Severity-routed human approval.** An Approval Gate classifies each validated patch as HIGH (mandatory human review), MEDIUM (a 120-second intervention window, then auto-approval), or LOW (auto-approval); Approve, Reject and diff-editing Modify decisions are all persisted as feedback.
3. **Real GitOps closure.** Approval opens a genuine GitHub pull request through a GitHub App, and a post-deploy probe watches for fresh fatal errors, invoking Argo CD rollback [6] to the prior known-good revision whenever recovery policy demands it.
4. **A Docker preview-deploy engine** that detects a cloned repository's stack, generates a missing Dockerfile, builds and runs the container under strict resource limits, health-checks it over HTTP and returns a live preview URL, feeding its failures back as first-class incidents.
5. **Intelligent role recommendation.** A heuristic engine over audit-log and login signal recommends privilege escalations and downgrades, each carrying an evidence-backed rationale that can be accepted or dismissed.
6. **A multi-tenant platform around the loop:** PostgreSQL Row-Level Security (RLS) [7], Role-Based Access Control (RBAC), MFA and SSO through WorkOS [12], Stripe billing [13], integrations with GitHub, Slack [17], Sentry [14], Datadog [16] and PagerDuty [15], a tamper-evident audit log, and an administrative console.

1.5 Scope of the Project

Under RBAC, Nexis serves three human roles:

- **Owner/Administrator** — provisions the organisation, connects integrations, sets recovery policy, decides on HIGH-severity approvals, manages members and billing, and acts on role recommendations.
- **Engineer/Member** — works daily with projects and incidents, watches live pipeline runs, and exercises Approve/Reject/Modify.
- **Viewer** — read-only visibility into incidents, recovery outcomes and cost, enforced through RBAC at the console level.

Beyond these roles, the **nine agents are also modelled as a first-class actor category**: the Sentinel raises incidents and the Validator renders verdicts much as a human actor would, under the same audit and policy constraints. This, in effect, narrows the human roles to supervisory decisions.

The delivered scope is the platform described above, evaluated end-to-end against a locally hosted Large Language Model through Ollama [20]. The `nexis.sh` domain has been registered, but a credentialed public production deployment falls outside this report's scope, as the limitations discuss honestly.

1.6 Significance of the Project

Nexis's significance lies in the closed loop itself: one continuous, durable and auditable workflow running from detection through to verified deployment, with the human reduced from a transport mechanism to a single, well-placed decision.

Chapter 6 evidences this live through two end-to-end recovery runs and an HTTP-verified preview-deploy of a Dockerfile-less public repository. In the first run, the pipeline correctly rejects the patch on approval-window timeout, the safety mechanism working as designed; in the second, it carries a fault from detection to an approved, deployed patch in under two minutes, with per-agent token and cost figures.

Chapter 2: Literature Review

2.1 Overview

Nexis is fundamentally a systems-integration project: established tools are composed into one closed loop that none forms alone, and the survey marks each tool as either a dependency or an extension.

Large Language Model (LLM) coding assistants. GitHub Copilot [1] completes code and drafts multi-file changes in the editor. It never observes a production incident, never validates against a failing system, and never deploys. Nexis’s Backend agent fills that same niche differently, invoked by a fault-detection pipeline rather than a keystroke, and machine-validated before any human sees it.

Durable workflow orchestration. Temporal [2] provides durable, replayable workflow execution, with retries and state persistence, but no engineering semantics. The nine-agent RecoveryPipeline runs on it as a Temporal workflow, so agent steps survive restarts as durable ActivityEvents. Role routing and contract-violation detection are Nexis’s logic, and so is the severity classification.

Causal inference and graph analysis. Neo4j [3] stores the code-dependency graph that the Pathfinder agent walks from symptom to candidate root cause, though evidence ranking is Nexis’s code. DoWhy [4] and its refutation tests serve as the sidecar’s methodological reference; Nexis, however, does *not* fit a DoWhy structural causal model, since no interventional production data exists. Instead, the sidecar ranks candidates by a documented proxy (evidence-chain specificity, graph proximity, textual overlap, scenario prior), together with a transparent per-component confidence breakdown.

The self-healing/AIOps ecosystem. Sentry [14] and Datadog [16] capture errors and aggregate telemetry, while PagerDuty [15] pages the on-call engineer; none of the three generates a fix, let alone validates one. All three feed Sentinel as webhook incident sources, and the Slack API [17] carries the reverse: one-click Approval-Gate approval or rejection of a validated patch.

Further dependencies. Argo CD [6] handles GitOps deploys, and its Rollback client is real, firing on a policy-gated SLO breach. Docker [10] backs the Validator’s network-isolated sandbox and preview deploys; Podman [11], the API-compatible evaluation substitute, behaved identically, reported honestly. Hypothesis [9] sees a novel use: the Quality Assurance (QA) agent generates pytest/Hypothesis suites from incident and diff, and a background loop replays them, raising incidents on pass-to-fail regressions. The Data Engineer agent, in turn, emits OpenLineage [8] RunEvents per migration. OpenTelemetry [5] and WorkOS [12] are used as documented, as are Stripe [13], Row-Level Security [7], pgvector [19], and Ollama [20], the LLM behind the report’s live evidence.

Table 2.1: Surveyed tools relative to Nexis.

Tool	Relation to Nexis
GitHub Copilot [1]	Backend-agent analogue only
Temporal [2]	Hosts RecoveryPipeline
Neo4j [3]	Dependency; Pathfinder traversal
DoWhy [4]	Reference; proxy used
OpenTelemetry [5]	Instrumentation dependency
Argo CD [6]	Deploy, rollback probe
Docker/Podman [10, 11]	Sandbox, deploy engine
Hypothesis [9]	Novel use: QA suites
OpenLineage [8]	Data Engineer conformance

Tool	Relation to Nexis
Sentry/Datadog/PagerDuty [14, 16, 15]	Webhook sources
Slack API [17]	One-click approvals

2.2 Research Gap

Industry already chains these tools routinely, yet none of them, alone or combined, both (a) models a software engineering *team* with explicit role specialisation and (b) closes the loop from automatic fault detection to a deployed, verified fix, with human judgement concentrated at one severity-routed decision point. Hence the break is always the same: between “a fault is known” and “a validated fix exists,” a human performs the diagnosis, authoring, testing, review, and release, unaided.

Nexis closes that span. Sentinel detects faults through an Exponentially Weighted Moving Average (EWMA) baseline breached at mean-plus-three-sigma, and the Synthesiser routes only the needed agents, the workflow visibly skipping the rest. The Architect’s `affected_files` contract is checked mechanically against the Backend diff; a violation forces HIGH severity, so no change auto-deploys unreviewed. QA generates property-based tests [9]; the Validator then executes patch and tests inside a sandbox, so no unvalidated patch reaches a human. Only one decision point engages: HIGH requires review, MEDIUM auto-approves after 120 seconds unless overridden, LOW passes immediately. Engineers may Approve, Reject, or Modify, and each decision is persisted for preference learning. Approval opens a real GitHub pull request under a post-deploy rollback probe [6].

Two properties set it apart from scripted tool-chaining. Role specialisation is structural: five Execution Team agents and four Self-Healing Loop agents each carry distinct inputs, outputs, and contracts under the Architect’s plan. Moreover, the loop closes over Nexis’s own infrastructure, since a preview deploy that fails its build or health check re-enters the pipeline as a first-class incident, so “write a working Dockerfile” becomes another ordinary repair task. To the author’s knowledge, no tool in common use combines both properties within a single system.

2.3 Conclusion

The landscape supplies nearly every ingredient, yet leaves a human to close the detection-to-repair loop. Thus, Nexis’s contribution is not a new algorithm but the composition of a role-specialised multi-agent team that detects, diagnoses, repairs, validates, and deploys autonomously, engaging human judgement once, at a severity-routed gate. The chapters that follow describe its specification, design, implementation, and testing.

Chapter 3: System Analysis and Requirements

3.1 Requirement Analysis

Nexis was built as a role-based, multi-tenant platform in which every datum belongs to exactly one organisation, and every request is evaluated against the caller’s organisation and role. Given this, the requirement analysis begins naturally with the actors sitting at the system boundary. Three human actor types were identified:

- **Owner/Administrator** — owns membership, roles, billing, policy, integrations, and the final human decision on high-stakes automated repairs.
- **Engineer/Member** — investigates incidents, exercises the evaluation tooling, works the recovery pipeline daily.
- **Viewer** — a read-only observer (stakeholder or auditor): inspects system state, never changes it.

The role model itself is *additive*: each tier holds every permission of the tier below it, so Section 3.2 lists only what each role *adds*. (Internally, this maps to owner/admin/member; Viewer is RBAC-gated on the frontend and API.)

A fourth actor here is not human at all: the **nine-agent self-healing fleet**. It acts autonomously inside the system boundary, detecting faults without being prompted and writing directly to the console’s own PostgreSQL, Neo4j, and object-storage stores, and its actions carry real consequences: GitHub pull requests, Argo CD rollbacks [6], preview redeployes. Roughly half of all incident, patch, and audit-table writes are agent-authored, so leaving the fleet out would misrepresent the system; modelling it instead makes the safety analysis explicit, since every Section 3.2.4 capability is bounded by a Section 3.3 control — sandboxing, severity classification, or the human approval gate.

Level 0/1 use case and data flow diagrams for all four actors appear in Sections 4.6–4.7.

3.2 Functional Requirements

3.2.1 Owner/Administrator can:

- Manage membership and roles, including the role-recommendation banner: accept applies under an optimistic-concurrency guard, dismiss starts a 30-day cool-down.
- Manage Stripe billing [13] and daily token budgets; export/verify the hash-chained audit log; get the daily digest; monitor System Health.
- Decide approvals (**Approve**, **Reject**, or **Modify**, where diff edits persist as RLHF feedback) and set recovery policy, including auto-rollback on SLO breach.
- Operate the preview-deploy engine; install GitHub App, Sentry [14], Datadog [16], PagerDuty [15], Slack [17] (in-message approve/reject), and Argo CD [6]; export evaluation CSVs.

3.2.2 Engineer/Member can:

- Authenticate via email/password (HttpOnly JWT cookie), magic link, or WorkOS SSO [12]; hashed, time-limited reset tokens; TOTP MFA; remote session revocation; scoped `nx_live_...` API keys.
- Create GitHub-connected projects; follow the nine-agent timeline over Server-Sent Events (SSE), inspecting per-agent status, tokens, cost, structured payloads.
- Trigger fault injection from 27 Live-Demo scenario cards (one wired end-to-end today, the rest “Coming soon”).
- Run the OpenAI-versus-Ollama evaluation harness (persisted transcripts, costs); query the pgvector knowledge base [19]; complete the post-incident NASA-TLX survey [18].

3.2.3 Viewer can:

- Sign in and read: dashboard KPIs (MTTR, success rate, open incidents, token spend), live incident timelines, and project, performance, agents, and activity pages — never mutating anything.

3.2.4 The Nine-Agent Fleet autonomously:

- **Detects** (Sentinel): a per-row fatal trigger plus an EWMA spike rule (mean + 3σ , Poisson floor, warm-up fallback).
- **Diagnoses** (Pathfinder): walks the Neo4j graph [3]; the causal sidecar ranks candidates by evidence specificity, proximity, overlap, and scenario prior [4].
- **Routes** (Synthesiser): assigns a scenario (regex fast path, LLM fallback) and an enforced `selected_agents` list.
- **Plans** (Architect): `plan_steps`, `affected_files`, `risk_level` form a contract — diffs beyond `affected_files` force HIGH severity, never auto-deploying.
- **Codes** (Backend) a minimal LLM-guided unified diff; **Tests** (QA) with auto-generated pytest/Hypothesis property suites [9], replayed continuously, regressions raising fresh incidents.
- **Packages** (DevOps): CI/CD and Argo CD manifests [6]; **handles data** (Data Engineer): paired forward/reverse SQL migrations, `information_schema` drift detection, OpenLineage events [8].
- **Validates** (Validator): every patch runs in a Docker sandbox [10] (no network, read-only filesystem, dropped capabilities, PID limit, `no-new-privileges`) before the approval gate.
- **Classifies severity**: HIGH (schema drift, sensitive paths, unknown scenarios, contract violations); MEDIUM (120-second auto-approve countdown); LOW (tiny UI-only diffs) auto-approves.
- **Deploys** (GitOps) via real GitHub pull requests; **rolls back** via Argo CD on post-deploy regression, policy permitting; **records** each step as a Temporal-durable SSE-streamed event [2]; and refiles deploy-engine build failures as incidents: broken builds self-heal too.

3.3 Non-Functional Requirements

Table 3.1: Non-functional requirements and realisation.

Requirement	Realisation in Nexis
Multi-tenancy	PostgreSQL Row-Level Security (RLS) on essentially every table [7]
Durability	Temporal-orchestrated workflows survive crashes and restarts [2]
Security	Sandboxed patch execution, encrypted patch storage, hashed tokens, hash-chained audit log
Scalability	Stateless Go services, queue-decoupled workers, parallel dependency probes
Responsiveness	Server-Sent Events stream the live incident timeline
Usability	Role-scoped console, one-click approvals, plain-language rationales

Isolation cannot depend on a developer remembering to add `WHERE org_id` to every query. Instead, RLS policies [7], bound per request to `app.current_org_id`, make the database itself refuse cross-tenant access, whether the request comes from a human or from one of the agents. A recovery run can span several minutes and often waits on a human decision, so pipeline state lives in Temporal [2] as journaled activities replayed deterministically after a crash — an in-memory orchestrator was rejected for exactly this reason, since it would simply strand incidents mid-repair. Generated code, moreover, is untrusted by construction: patches execute only sandboxed, patch blobs are encrypted at rest, tokens exist only as SHA-256 hashes (30-minute TTL, non-leaky 202), and audit-log tampering detectably breaks the chain.

3.4 System Feasibility

3.4.1 Technical Feasibility

Nexis was deliberately built as a polyglot system. The four independent backend services are written in Go, for static binaries, cheap concurrency, and a first-class Docker API client on which the webhook and container work depends. The console, however, is Next.js/React/TypeScript, chosen for its role-gated, SSE-streaming, Monaco-editing interface, while the causal sidecar runs on Python/FastAPI, on the scientific-Python/Hypothesis [9] stack. PostgreSQL 17 with pgvector [19] supplies transactions, RLS, and vector retrieval in one engine, whereas Neo4j [3] holds the dependency graph because diagnosis is graph traversal, which relational queries express poorly. Temporal [2] was proven by both live recovery runs in Chapter 6, and the whole system was built and verified on one development machine, with Podman [11] substituting for Docker [10].

3.4.2 Economic Feasibility

Every dependency Nexis relies on is open source, namely PostgreSQL, Neo4j Community, Redis, MinIO, Temporal, OpenTelemetry [5], and Grafana, so there is no licence cost. The one significant cost is LLM inference, handled as a dual path: OpenAI models for production, local Ollama [20] models (`gpt-oss:20b`, `qwen2.5-coder:14b`) for development. Every live run in this report in fact used the local provider, so the complete system was exercised at zero token cost; the cost tracker and daily budgets bound the paid path.

3.4.3 Operational Feasibility

On logging in, administrators see organisation-wide KPIs, approvals, and system health; engineers see incidents and the live pipeline; viewers see the same picture, read-only. High-stakes autonomy, in this regard, reduces to a single click, Approve, Reject, or Modify, optionally from Slack [17], while severity classification lets low-risk fixes pass unattended but never the sensitive ones. The built-in NASA-TLX instrument [18] lets the operator-workload claim eventually be measured rather than simply asserted; the intended operator, furthermore, is an ordinary engineer with a browser, not a dedicated operations team.

Chapter 4: System Design and Architecture

The design of Nexis is presented across four complementary levels in this chapter: the structural view (Sections 4.1–4.5), the functional view through use case diagrams (4.6), the data view through Data Flow Diagrams (4.7), and a UML sequence diagram tracing one complete recovery run (4.8). Since the self-healing loop is the central contribution of the project, it has been given the most space of the four.

4.1 System Context

Figure 4.1 places Nexis at the centre of its environment. Three human roles sit around it (Owner/Administrator, Engineer/Member, Viewer), the nine-agent fleet acts as a fourth, autonomous actor, and a wider ring of external systems completes the picture: GitHub, Slack [17], the LLM provider (OpenAI or Ollama), Argo CD, Stripe [13], the alert sources Sentry [14], Datadog [16], and PagerDuty [15], WorkOS SSO [12], and SES/SMTP mail.

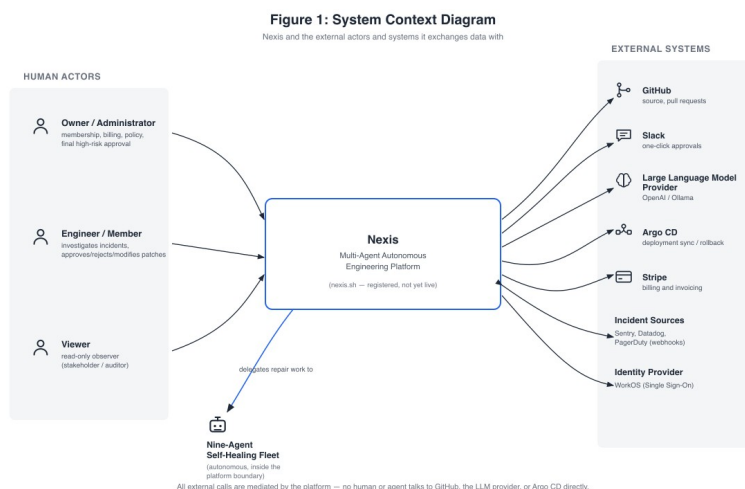


Figure 4.1: System Context Diagram. (Nexis and the external actors and systems it exchanges data with).

Every entry point, whether the web console, webhooks, Slack callbacks, or a plain HTTP client, converges on the control-plane; no human or agent talks to a data store or an external integration directly. It should be disclosed plainly that the `nexus.sh` domain is registered, but the platform is not yet publicly live (Section 4.5).

4.2 Component Architecture

Figure 4.2 opens up the platform boundary. The Next.js 16 web console sits to one side; moreover, the centre is anchored by a single Go control-plane (`:8080`) that hosts the chi-routed API layer (authentication, RBAC, RLS binding, rate limiting), the always-on Sentinel detector (EWMA $+3\sigma$ spike rule), and the durable Temporal RecoveryPipeline workflow [2], which orchestrates the nine-agent loop.

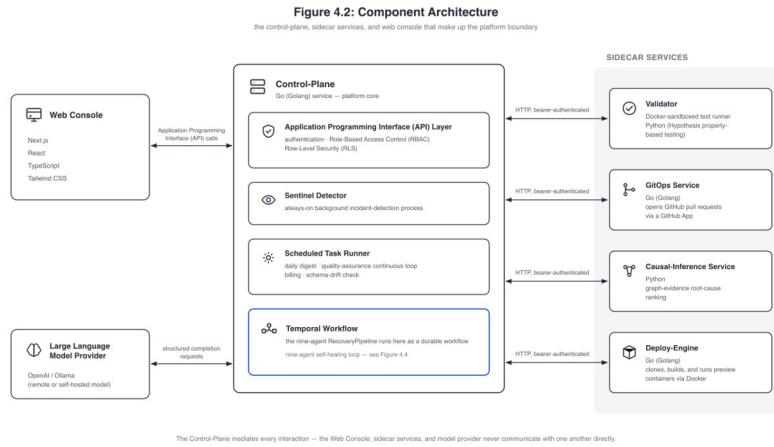


Figure 4.2: Component Architecture. (the control-plane, sidecar services, and web console that make up the platform boundary).

Four bearer-authenticated sidecars sit around it: the Docker-sandboxed Validator (:8081) [10], the GitOps PR service (:8082), the Python/FastAPI causal-inference ranker (:8090, an honest statistical proxy rather than a fitted structural causal model [4]), and the deploy-engine (:8091), together with an Argo CD Sync/Rollback client [6]. The LLM spine, for its part, pairs OpenAI (gpt-4o family) with Ollama [20] (gpt-oss:20b, qwen2.5-coder:14b) as a fully supported alternative, backed by pgvector retrieval [19] and per-agent JSON Schema validation with retry; every interaction, in turn, is mediated by the control-plane.

4.3 Data and Storage Architecture

The four storage systems, along with Temporal’s workflow persistence layer, are laid out in Figure 4.3.

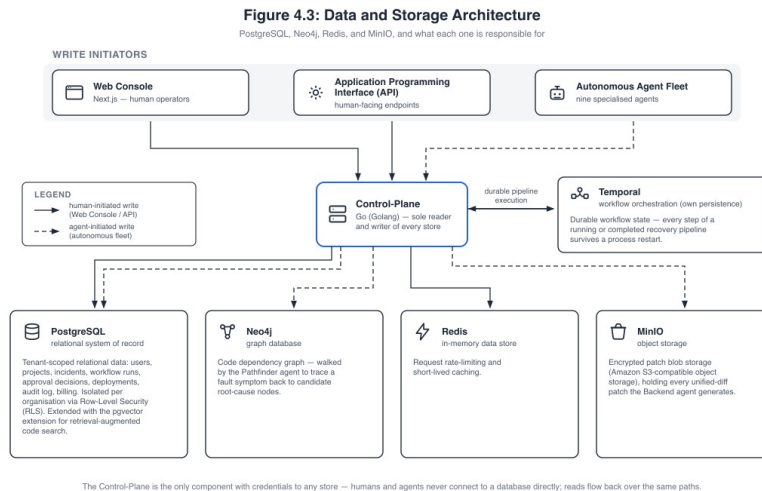


Figure 4.3: Data and Storage Architecture. (PostgreSQL, Neo4j, Redis, and MinIO, and what each one is responsible for).

PostgreSQL 17 with pgvector serves as the system of record, made multi-tenant via Row-Level Security [7] across 33 migrations; Neo4j [3] holds each repository’s dependency graph for Pathfinder; Redis takes care of rate limiting and caching; and MinIO functions as the encrypted patch store. The key point is isolation, as stated in the figure’s footer: only the control-plane holds store credentials, and neither humans nor agents ever connect to a database directly.

4.4 Self-Healing Loop Pipeline

Figure 4.4 presents the central contribution of the project as a single flow, the nine agents running in order: Sentinel (detect), Pathfinder (localise via the Neo4j graph), Synthesiser (classify, select agents), Architect (plan and file-level contract), Backend (minimal patch), Quality Assurance (property-based tests), DevOps running in parallel with Data Engineer (manifests and migrations), and finally Validator (sandboxed shadow-execution).

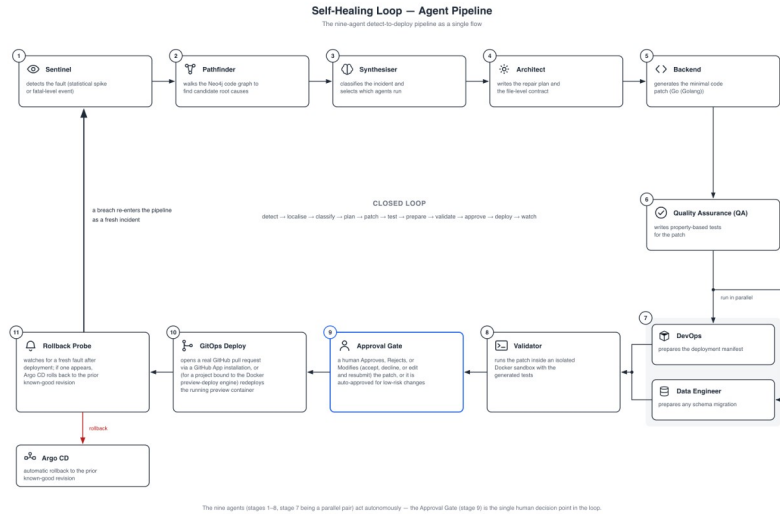


Figure 4.4: Self-Healing Loop — Agent Pipeline. (the nine-agent detect-to-deploy pipeline as a single flow).

A person enters the loop only at this point: the Approval Gate (approve, reject, or modify; auto-approved for low-risk changes) forms the single human decision point in the whole pipeline. GitOps Deploy consequently opens a real pull request or redeploys a preview container, while the Rollback Probe rolls back via Argo CD on a fresh post-deploy fault and feeds the breach straight back into detection.

4.5 Deployment and Infrastructure

Figure 4.5 lays out how the platform is built, delivered, and observed: GitHub Actions handles CI, Docker (Podman-compatible) serves as the universal runtime, Argo CD carries out GitOps delivery, and Terraform declares the AWS infrastructure as code.

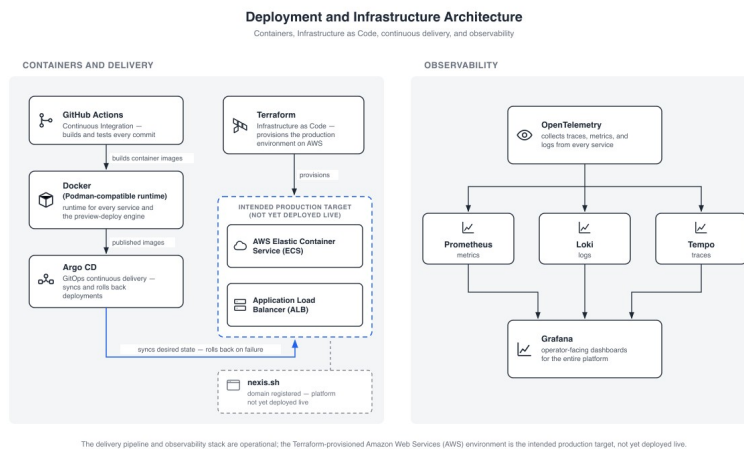


Figure 4.5: Deployment and Infrastructure Architecture. (containers, Infrastructure as Code, continuous delivery, and observability).

This diagram, too, is deliberately honest about status: the AWS ECS cluster behind an Application Load Balancer is the *intended* production target declared in Terraform, not yet deployed live, and the registered `nexis.sh` domain does not yet serve the platform. Observability follows OpenTelemetry [5] into Prometheus, Loki, and Tempo, all of it visualised in Grafana. The evaluation environment itself ran the sandbox and deploy-engine on Podman [11], which put its Docker-API compatibility to practical use.

4.6 Use Case Diagrams

4.6.1 Use Case Diagram, Level 0

The coarsest functional view appears in Figure 4.6: the three human actors and the autonomous nine-agent fleet set against a single system boundary, with the mediated external systems placed to the right.

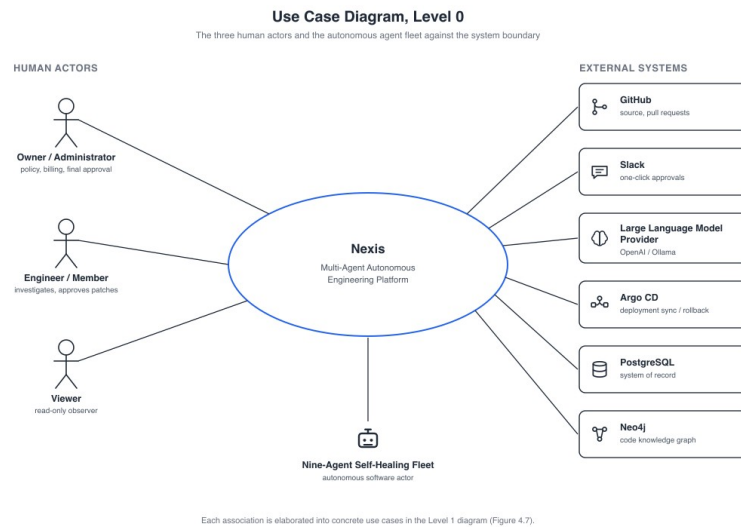


Figure 4.6: Use Case Diagram, Level 0. (the three human actors and the autonomous agent fleet against the system boundary).

Mediation is the essential property here: every association terminates at the system boundary, and no human ever calls GitHub, the LLM provider, or Argo CD directly. The agents act on the external systems on the humans' behalf instead, surfacing back to a person only at the approval gate.

4.6.2 Use Case Diagram, Level 1

Figure 4.7 expands the Level 0 bubble into concrete use cases, grouped by minimum role, where generalisation arrows make the role inheritance explicit.

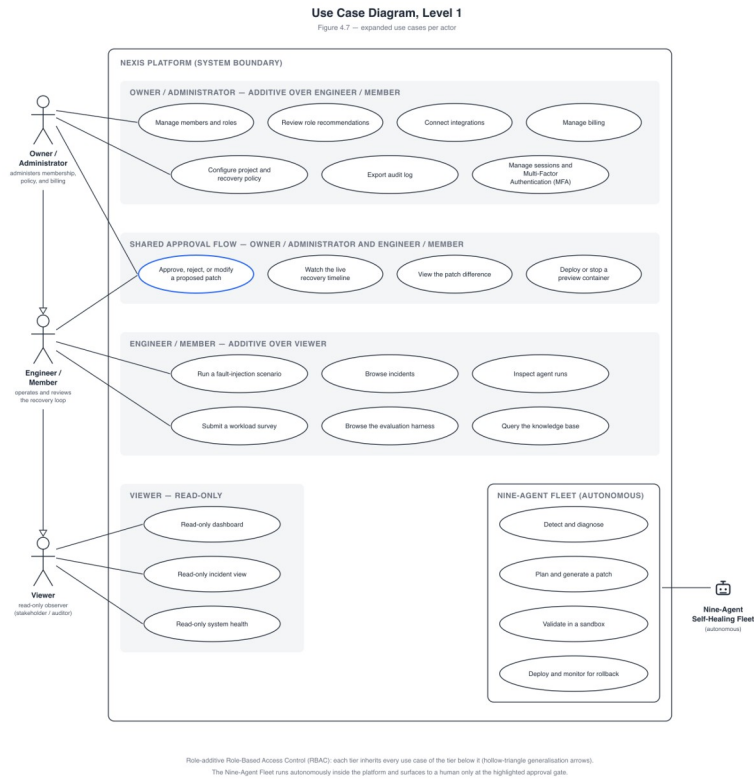


Figure 4.7: Use Case Diagram, Level 1. (expanded use cases per actor).

The administrative tier holds fifteen use cases in total (membership, role recommendations, integrations, recovery policy and kill switch, billing, the RLHF/JSONL feedback export, audit, MFA). Four approval-flow use cases are shared with the Engineer/Member, whose own tier includes demo scenarios, agent runs, and the NASA-TLX survey [18]; the Viewer holds exactly three read-only use cases. The model is role-additive, in that each tier inherits every use case of the tier below it. As such, the read-only Viewer experience is enforced through RBAC gating in the console, even though the database role enumeration itself is only `owner/admin/member`.

4.7 Data Flow Diagrams

4.7.1 Data Flow Diagram, Level 0 (Context)

At the context level, Figure 4.8 shows the entire platform as a single process 0 with three top-level stores: D1 the tenant database (PostgreSQL, under Row-Level Security), D2 the code graph (Neo4j), and D3 the encrypted patch store (MinIO).

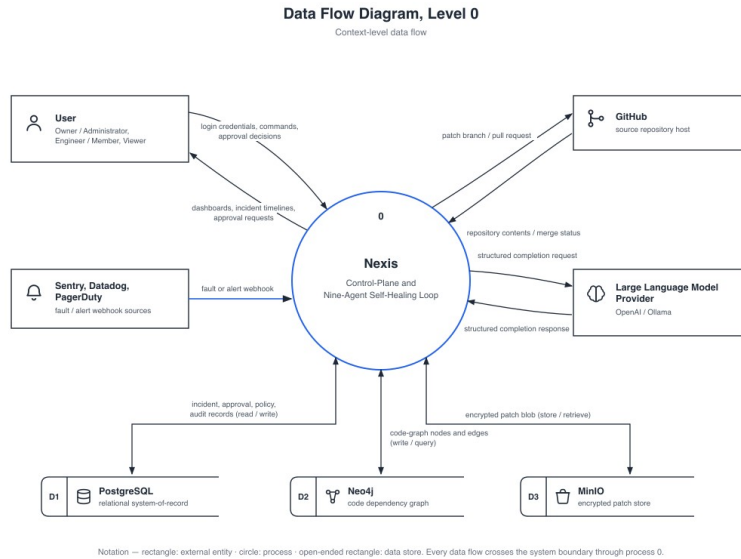


Figure 4.8: Data Flow Diagram, Level 0. (context-level data flow).

Four entity flows cross the boundary: the user, GitHub, the alert sources (producers of incidents, not consumers), and the LLM provider. Every one of these is mediated by process 0, since there is no direct path from a human to an integration.

4.7.2 Data Flow Diagram, Level 1 — Owner/Administrator

Figure 4.9 decomposes the administrative surface into ten numbered processes, spanning identity and access, platform administration, and incident and approval operations, all over the tenant-scoped PostgreSQL stores; Table 4.1 summarises the most essential of these.

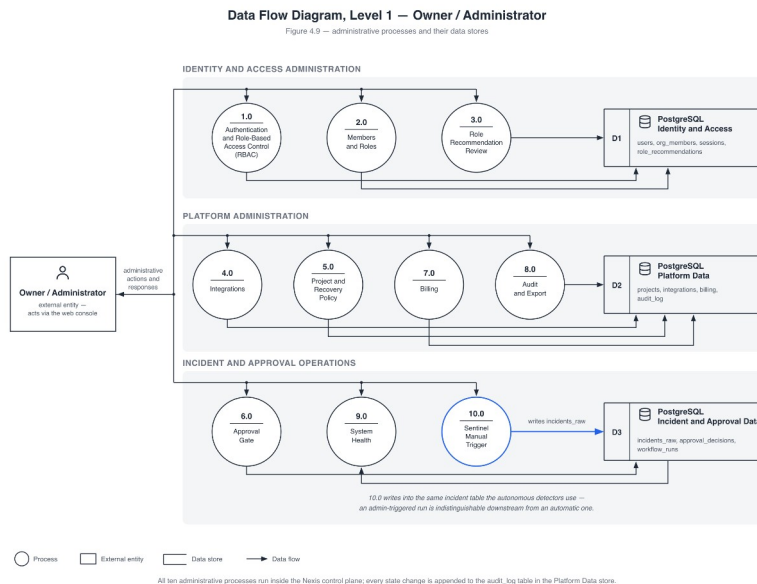


Figure 4.9: Data Flow Diagram, Level 1 — Owner / Administrator. (administrative processes and their data stores).

Table 4.1: Owner/Administrator Data Flow Diagram Level 1 processes and their primary data stores.

Process	Responsibility	Primary store written
1.0 Authentication and RBAC	login, JWT session issue, role checks	users, sessions
5.0 Project and Recovery Policy	recovery policy, SLO, kill switch	projects
6.0 Approval Gate	approve / reject / modify decisions	approval_decisions
8.0 Audit and Export	hash-chained audit writes, CSV export	audit_log
10.0 Sentinel Manual Trigger	administrator-initiated detection run	incidents_raw

The most important flow is 10.0: a manual detection run writes into the *same* incidents_raw table as the automatic detectors, with no separate code path. Hence, every autonomous-path guarantee (durability, validation, severity routing, auditability) holds for manual runs as well, at no extra cost. Process 5.0 carries the kill switch that suspends autonomous recovery, and every state change lands in the hash-chained audit_log.

4.7.3 Data Flow Diagram, Level 1 — Self-Healing Loop

Twelve numbered processes make up the loop in Figure 4.10’s decomposition: detection and decision, the Layer-1 execution team, and the validation–approval–deployment–verification tail, spread over five stores (D1 incidents, D2 code graph, D3 patch store, D4 approvals and feedback, D5 deployments and lineage).

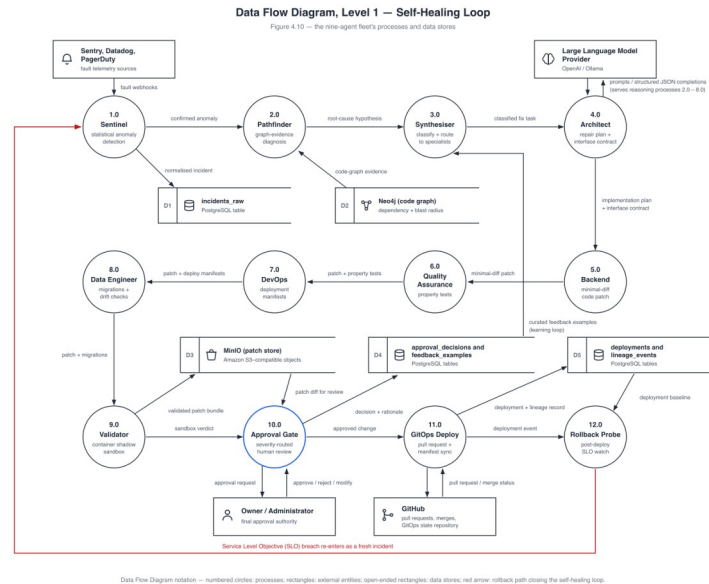


Figure 4.10: Data Flow Diagram, Level 1 — Self-Healing Loop. (the nine-agent fleet’s processes and data stores).

The closing annotation states the property that gives the project its very title: the loop is closed. An SLO breach seen by the rollback probe (12.0) lands in incidents_raw as a fresh fatal and re-enters at Sentinel (1.0), with an Argo CD rollback to the prior known-good revision whenever RollbackOnSLOBreach is set. Thus, recovery, verification, and re-detection all form one cycle over a single shared data path.

4.8 Self-Healing Loop Sequence Diagram

One complete recovery run is traced in Figure 4.11 across all nine agent lifelines, in eighteen numbered messages. Each message is a named Temporal activity, durably recorded and streamed to the incident timeline over Server-Sent Events, in the exact order that the Chapter 5 screenshots show.

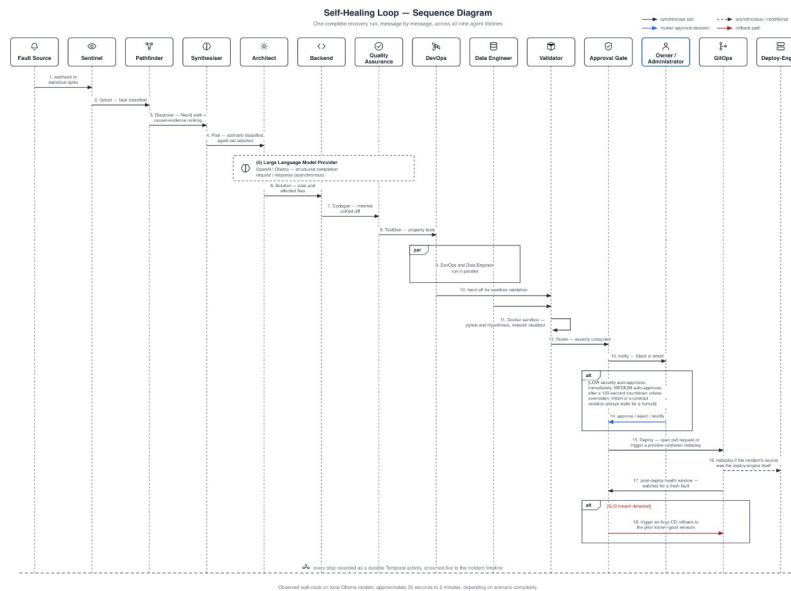


Figure 4.11: Self-Healing Loop — Sequence Diagram. (one complete recovery run, message by message, across all nine agent lifelines).

No unvalidated patch is allowed past the sandboxed Validator (`--network=none`, `--read-only`, `--cap-drop=ALL`, `pytest/Hypothesis` suite [9]). Severity routing then forces an explicit human decision for HIGH: schema drift, sensitive-path diffs (`*.sql`, `migrations/`, `auth/`, `security/`, `crypto/`), unknown scenarios, or a contract violation (a diff falling outside the Architect’s `affected_files`), whereas LOW auto-approves and MEDIUM auto-approves after 120 seconds. One of the two live runs in Chapter 5 genuinely timed out at this gate, the safety mechanism working as designed. Observed wall-clock time ran from approximately 35 seconds to 2 minutes on local Ollama models, a figure corroborated by Chapter 5’s fully successful run of 1 minute 40 seconds.

Taken together, these views describe one coherent design: a mediated boundary, a control-plane serving as the sole gateway to every component and store, a role-additive human surface set beside an autonomous fleet, a single shared data path for both manual and automatic operation, and a recovery cycle that feeds its own failures back into itself. Chapter 5 goes on to present the implementation, while Chapter 6 supplies the evidence that the loop actually closes in practice.

Chapter 5: Implementation

The system covered here is the one that actually runs, not a proposed version of it: the technology stack, a screen-by-screen tour of the platform, and the database schema underneath the multi-tenant control plane. Every screenshot is an unretouched capture from the running development environment, and errors or pending states are reported as they happened, not swapped for a tidier run.

5.1 Tools and Technology

Nexis is built as a polyglot pnpm + Turborepo monorepo, with each subsystem implemented in whichever language best suits its role:

- **Backend:** Go 1.25 powers four modules: `services/control-plane` (multi-tenant API), `services/validator` (sandboxed patch validation), `services/gitops` (GitHub-App deployment), and `services/deploy-engine` (Docker preview deploys). A Python/FastAPI sidecar, `services/causal-inference` (port 8090), ranks candidate root-cause nodes for the Pathfinder agent, while Temporal [2] durably orchestrates the nine-agent RecoveryPipeline, with every step logged as an ActivityEvent so crashed workers resume in place.
- **Frontend:** Next.js 16, React 19, and TypeScript make up `apps/web`, styled with Tailwind CSS 4 and Radix UI. Monaco handles unified-diff editing, Recharts renders the charts, and Server-Sent Events keep incident pages live without resorting to polling.
- **Data:** PostgreSQL 17 serves as the system of record (Row-Level Security (RLS) [7], 33 migrations); pgvector [19] backs knowledge-base retrieval; Neo4j [3] holds the code dependency graph; Redis handles rate limiting and caching; MinIO stores S3-compatible encrypted patch blobs; and Temporal keeps its own durable event history.
- **AI / LLM:** OpenAI (GPT-4o family) is the production-configured provider, while Ollama [20] (`gpt-oss:20b`, `qwen2.5-coder:14b`) is fully supported; in fact, all live-run evidence in this chapter used Ollama end-to-end, with no cloud Large Language Model (LLM) credentials involved. The Quality Assurance (QA) agent generates Hypothesis [9] property-based tests, which the Validator then executes inside its sandbox. It may be noted that the causal-inference sidecar is a statistical/graph ranking algorithm rather than a fitted DoWhy [4] model, since that would require interventional production data that does not yet exist, a limitation stated openly.
- **DevOps:** Docker API [10] serves both the Validator sandbox and preview deploys and is Podman-compatible [11]; in the evaluation environment, Podman stood in for it and worked identically. Terraform provisions AWS ECS behind a load balancer, OpenTelemetry [5] feeds Prometheus, Loki, Tempo, and Grafana, and Argo CD [6] backs the DevOps agent's Sync/Rollback client.
- **Version control:** Handled via Git and GitHub, with a GitHub App installation supplying short-lived tokens for cloning and for the GitOps blob → tree → commit → ref → Pull Request deployment path.

5.2 Project View

The captures that follow are ordered the way a new organisation would encounter the platform.

5.2.1 Authentication and Onboarding

Access relies on a local JSON Web Token (JWT) session, kept in an HttpOnly cookie, with optional WorkOS [12] Single Sign-On available on top of it. The sign-up screen (Figure 5.1) creates the account, and the forgot-password flow issues a 32-byte reset token, SHA-256-hashed with a 30-minute time-to-live; it returns the same 202 response regardless of whether the e-mail address actually exists, hence no account information leaks out.

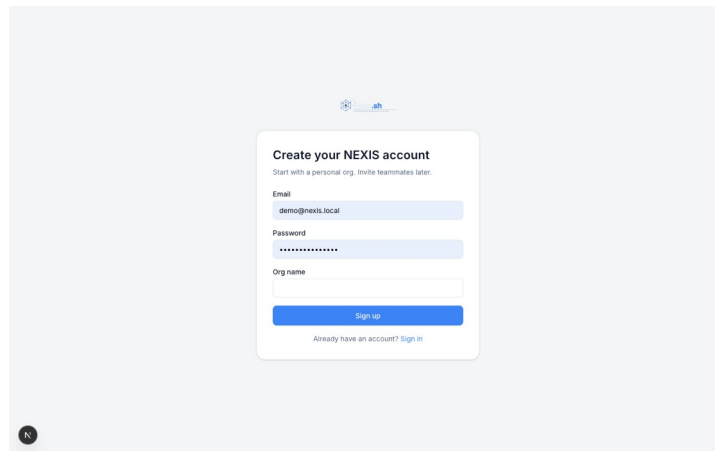


Figure 5.1: Sign-Up Screen. (new account registration).

5.2.2 Administrator / Owner Console

Once the administrator logs in, the console home (Figure 5.2) presents KPI tiles covering Mean Time To Recovery (MTTR), recovery success rate, open incidents, and mean tokens per run; alongside these sit an incidents-over-time chart, a system status panel, a projects health grid, and an activity feed.

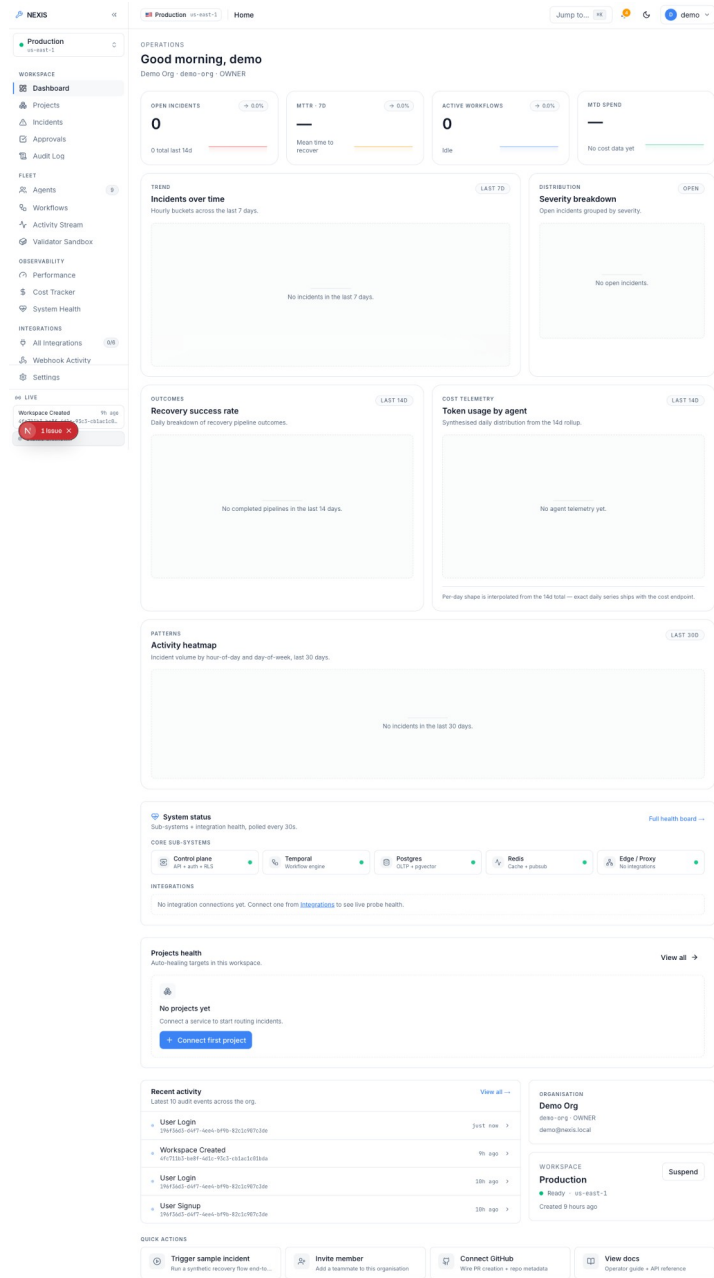


Figure 5.2: Administrator Dashboard. (KPI tiles, incidents-over-time chart, and system status panel).

Moving to the Agents page (Figure 5.3), the nine-agent fleet is listed across both layers: the five Execution Team agents (Architect, Backend, QA, DevOps, Data Engineer) and the Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator). Every ingested incident, in turn, appears on the Incidents page (Figure 5.4).

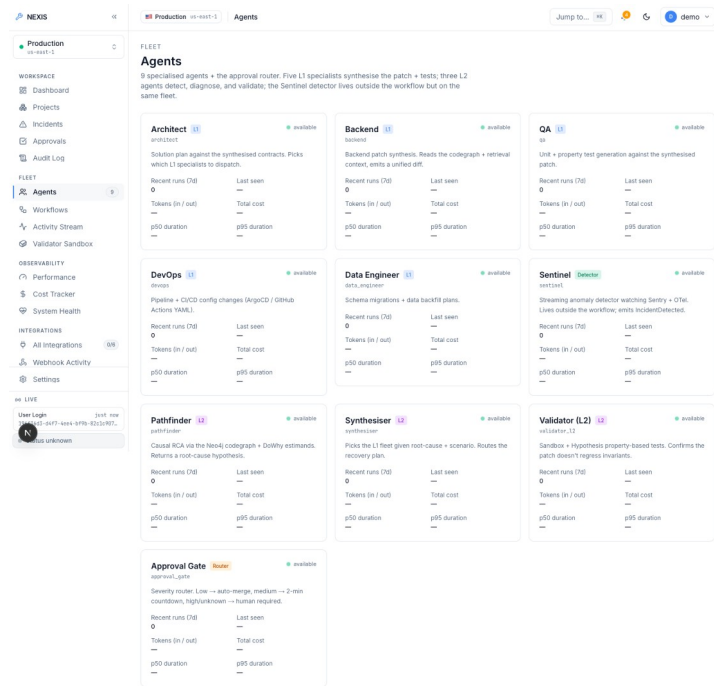


Figure 5.3: Agents Screen. (the nine-agent fleet roster across both layers).

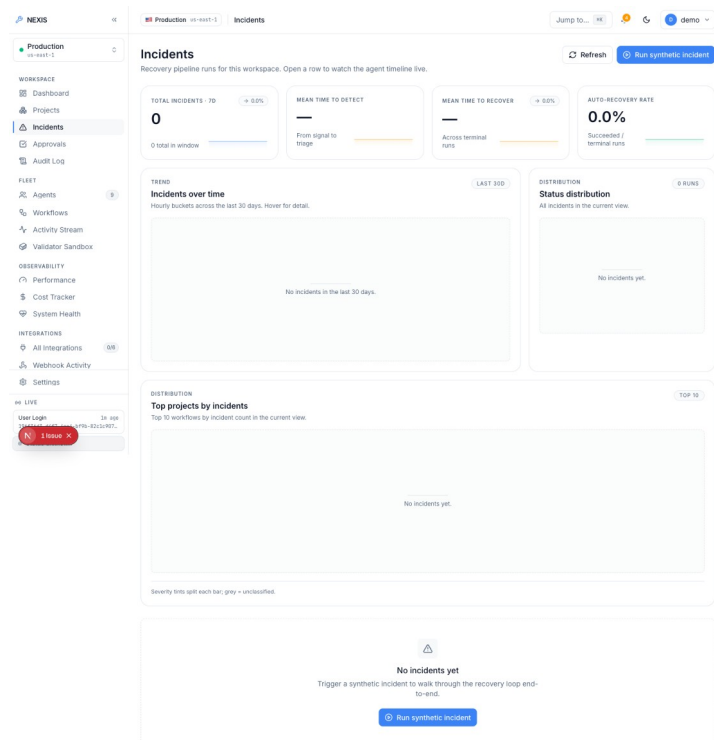


Figure 5.4: Incidents Screen. (the organisation-wide incident list).

Under Settings sit member management with `owner` / `admin` / `member` roles, a genuine heuristic role-recommendation panel offering evidence-backed rationales with a 30-day cool-down on dismissal, remotely revocable sessions, scoped API keys (`nx_live_...`), and Stripe billing [13].

5.2.3 Self-Healing Loop — Live Evidence

This subsection carries the evidentiary weight: two real, end-to-end executions of the nine-agent RecoveryPipeline, run against a live local LLM (Ollama, qwen2.5-coder:14b and gpt-oss:20b), are shown exactly as they occurred, one safely auto-rejected at the approval gate and the other approved and fully succeeded. The Live Demo entry point offers 27 fault-scenario cards, of which exactly one is fully wired end-to-end today; the rest remain UI-complete but backend-pending, each labelled “Coming soon”. Both runs used the wired path, and Figure 5.5 shows the pipeline structure.

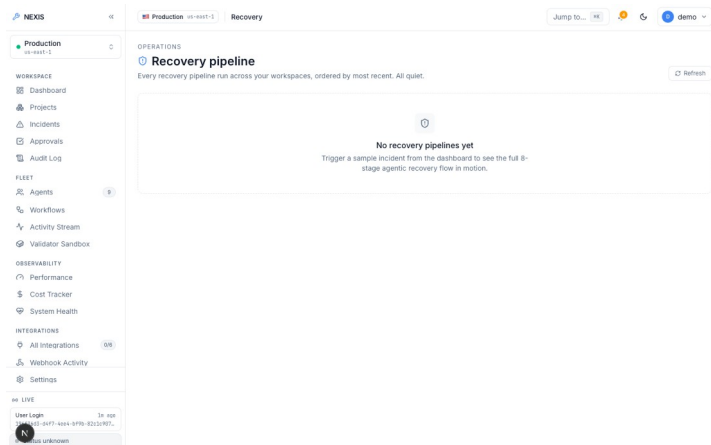


Figure 5.5: Recovery Pipeline Screen. (the nine-agent pipeline structure).

Figure 5.6 catches the first run genuinely mid-flight, with Architect, Backend, and QA already completed and showing real, non-zero token counts and costs from actual LLM calls, while DevOps and Data Engineer are still running in parallel. A stubbed pipeline would report zero tokens; consequently, these figures are themselves evidence.

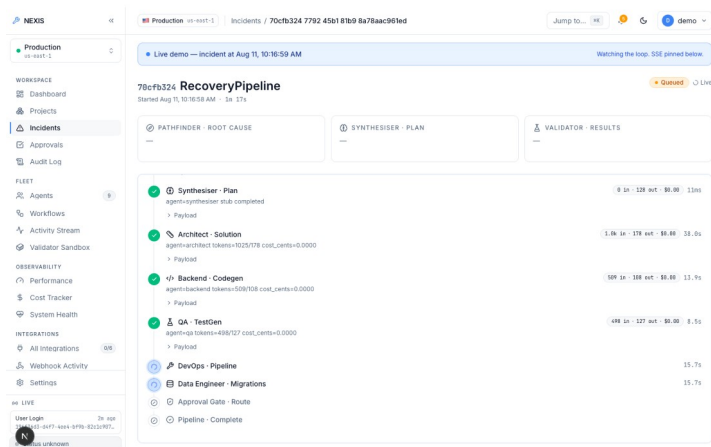


Figure 5.6: Incident Detail, Mid-Flight. (completed Architect/Backend/QA steps with real token counts; DevOps and Data Engineer running in parallel).

Every step then completed green, leaving severity at “Medium” and approval at “Pending” inside a 120-second window. That window elapsed with no decision, and the workflow terminated as `timeout_rejected` — an honest negative result, reported deliberately: absent a human decision, the system refuses to deploy unreviewed code rather than proceeding.

The second run supplies the positive counterpart to this result: approved within the window, it records “Medium” severity, “Approved”, “Succeeded”, and a full nine-step timeline completing in 1 minute 40 seconds, with real per-agent token and cost figures throughout (Figure 5.7).

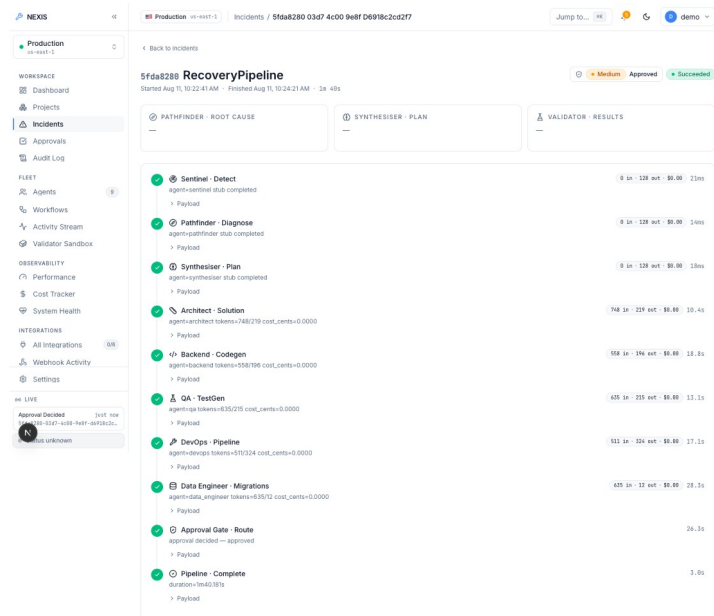


Figure 5.7: Fully Approved Run. (Medium severity, Approved, Succeeded; complete nine-step timeline in 1 minute 40 seconds).

Pending runs surface in the Approvals queue (Figure 5.8), where three human actions are available (Reject, Modify, Approve), each decided in the dialog shown in Figure 5.9. Modify is where the Reinforcement Learning from Human Feedback (RLHF) mechanism becomes concrete: the engineer edits the LLM-generated unified diff before approving it, and that edited diff, along with every terminal decision, is persisted to `feedback_examples`, feeding a JSONL export for future fine-tuning. Submitting a paid fine-tuning job, however, remains a deliberate manual step rather than an automated one.

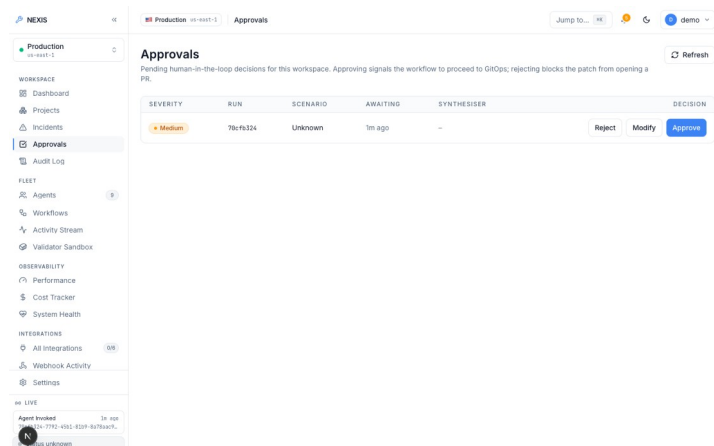


Figure 5.8: Approvals Queue. (real pending approval row with Reject / Modify / Approve actions).

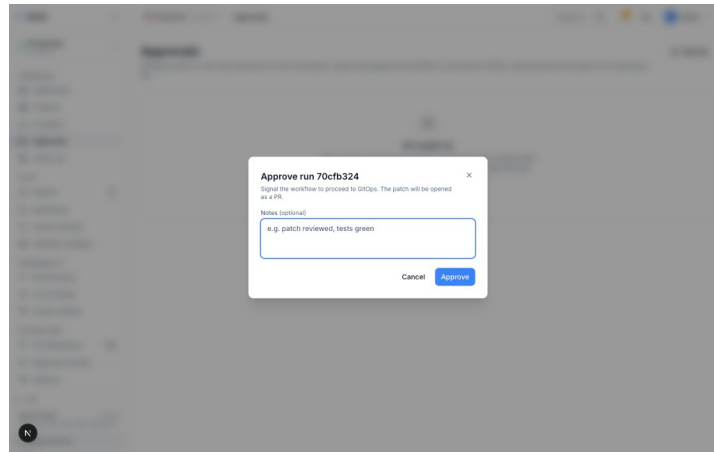


Figure 5.9: Approval Decision Dialog. (the Reject / Modify / Approve decision surface for a pending patch).

The patch itself is genuinely model-generated: the Backend step’s raw JSON records the unified diff produced by qwen2.5-coder:14b with provider="ollama", parsed model output, not a fixture template. The Backend agent’s own detail page appears in Figure 5.10.

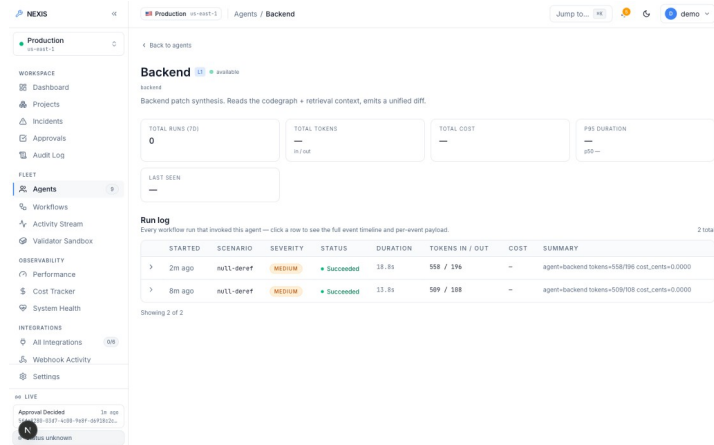


Figure 5.10: Agent Detail, Backend Agent. (per-agent configuration and run history).

5.2.4 Docker Preview-Deploy Engine (Ops Tab)

Deploy action, live status, preview URL, logs, and history all live on the project’s Ops tab, which fronts the preview-deploy engine. Attempting Deploy on a project with no GitHub App connection (Figure 5.11) returned the real validation error “project has no github repository bound”, surfaced verbatim. This is *not* a successful deploy; rather, it is the validation chain correctly rejecting a request it cannot fulfil, and the limitation is environmental (no GitHub App credentials in the evaluation environment) rather than a defect in the engine.

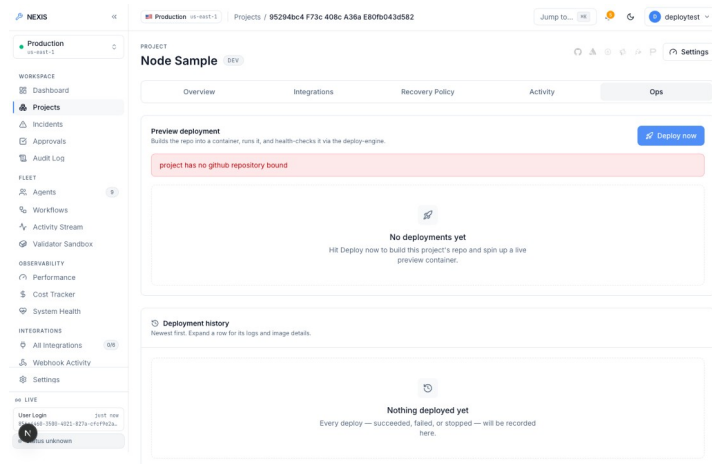


Figure 5.11: Project Ops Tab, Deploy Attempt. (a real validation error — “project has no github repository bound” — returned by the live API).

Independently of that limitation, the engine was proven live (Listing 5.1): `heroku/node-js-getting-started`, which ships without a Dockerfile, was cloned, detected as Node.js, handed a generated Dockerfile, built, and run under resource limits, and the preview URL served the app’s real rendered HTML before a clean stop and removal.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
  "status": "running", "dockerfile_source": "generated", "detected_stack": "node",
  "url": "http://localhost:54671", "image_tag": "nexus-preview-test-project:22d06076c357"

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status":"stopped"}; container removed.
```

Listing 5.1: Verified live deploy-engine run.

A failed build or a failed health check inserts a real incident, with `source: "deploy_engine"`, into the very same Sentinel-to-RecoveryPipeline path used by every other incident source.

5.2.5 Integrations and Observability

External providers are managed from the Integrations page (Figure 5.12): the GitHub App (installation, repository listing, webhook-driven incidents on merged pull requests); Sentry [14], Datadog [16], and PagerDuty [15], each a webhook incident source; Slack [17], with OAuth install and one-click approve/reject from a Slack message; Argo CD; and Stripe. Companion pages log webhook deliveries and probe integration health.

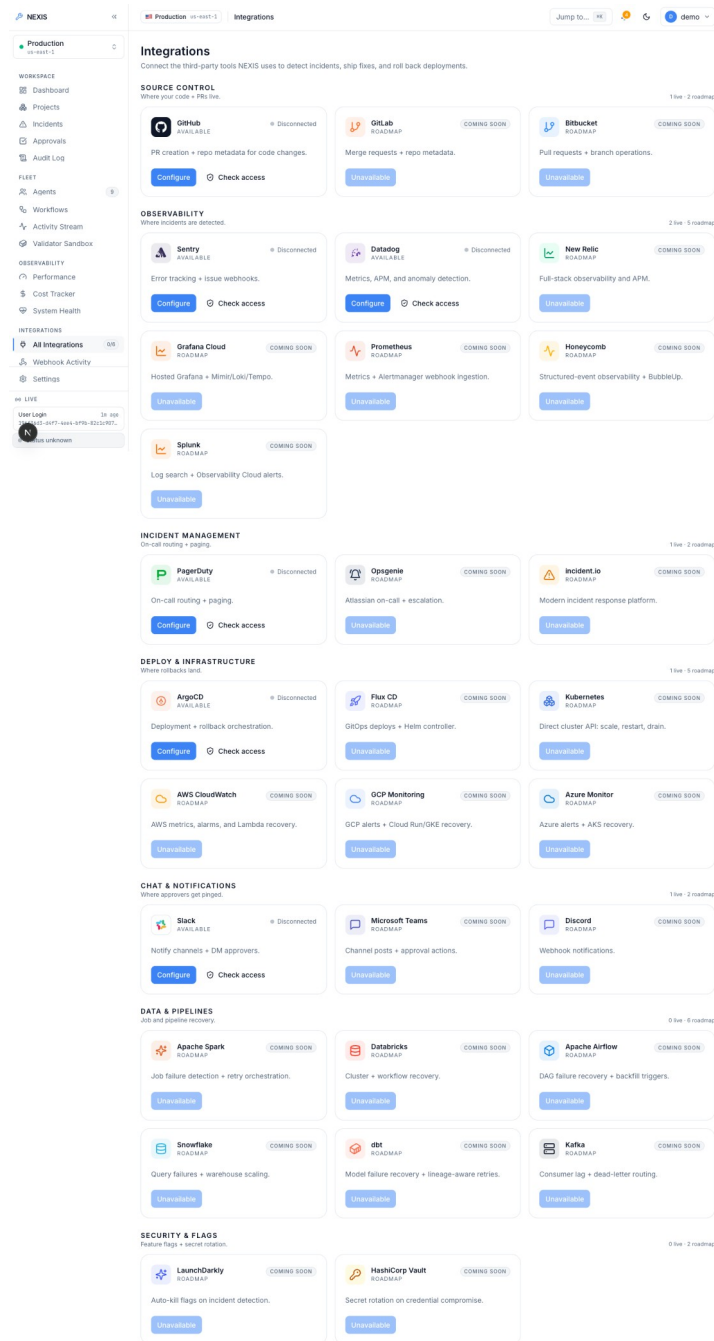


Figure 5.12: Integrations Screen. (provider catalogue: GitHub App, Sentry, Datadog, PagerDuty, Slack, Argo CD, Stripe).

On the observability side sit performance charts, a per-organisation token cost tracker with a daily budget, parallel system-health probes (PostgreSQL, Redis, Neo4j, MinIO, Temporal), and a pgvector-backed knowledge base. The Eval Harness (Figure 5.13) runs side-by-side OpenAI-versus-Ollama comparisons across fixture incidents, persisting transcripts, tracking cost, and offering administrator-only CSV export.

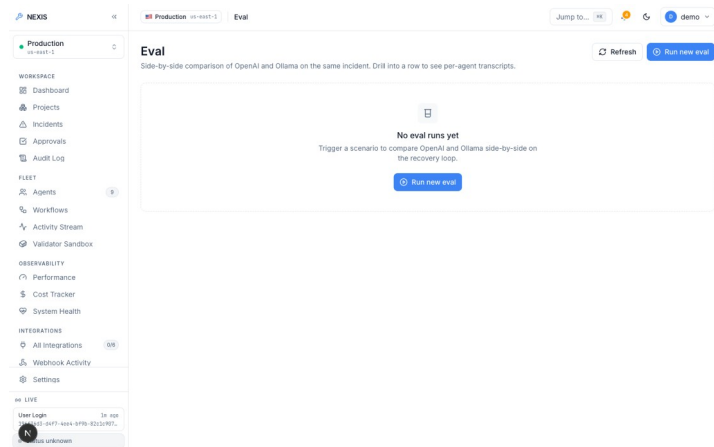


Figure 5.13: Eval Harness. (side-by-side OpenAI-versus-Ollama comparison over fixture incidents).

Append-only and hash-chained, the Audit Log (Figure 5.14) builds each entry on its predecessor’s hash; thus, tampering breaks the chain and becomes detectable through the verify endpoint. CSV export is also available for offline review.

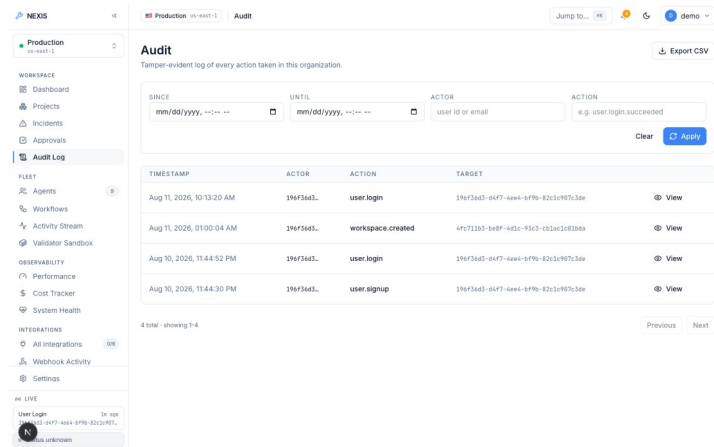


Figure 5.14: Audit Log. (append-only, hash-chained entries with CSV export).

5.3 Database Schema

State lives in PostgreSQL 17, spread across 33 sequential migrations. Multi-tenancy is enforced at the database level itself: nearly every table carries an organisation identifier, RLS policies [7] restrict queries to the calling organisation’s rows, and a request-scoped transaction binds `app.current_org_id`. As such, even a query omitting an organisation `WHERE` clause cannot touch another tenant’s data. Two exceptions are deliberate: `users` remains a global identity, with tenancy applied through `org_members`, while `sessions` stays user-scoped — a login session belongs to a person, not a tenant. The most important tenant-scoped tables are set out in Table 5.1.

Table 5.1: Key tenant-scoped tables in the Nexis PostgreSQL schema.

Table	Purpose	RLS scope
<code>incidents_raw</code>	Every ingested fault event, from webhooks, fixtures, and the deploy engine	Organisation-scoped
<code>workflow_runs</code>	One row per Temporal RecoveryPipeline execution, with per-	Organisation-scoped

Table	Purpose	RLS scope
	step activity events	
approval_decisions	Terminal approval-gate outcomes: approve, reject, modify, timeout	Organisation-scoped
feedback_examples	RLHF training rows persisted from every terminal approval decision	Organisation-scoped
audit_log	Append-only, hash-chained record of privileged actions	Organisation-scoped

Neo4j holds the dependency graph that the Pathfinder traverses, MinIO the encrypted patch blobs referenced from `workflow_runs`, and pgvector columns back the knowledge-base retrieval; Temporal's durable event history, moreover, can reconstruct the per-step records in `workflow_runs` in a recovery scenario.

Chapter 6: Testing and Results

This chapter sets out the verification evidence as it was gathered: static checks across every service, live end-to-end runs driven by a real local Large Language Model (LLM) rather than stubs or mocks, and the defects that verification turned up.

6.1 Static Verification

Every command in Table 6.1 was executed against the final codebase; the result column is not aspirational. The Go suites include the JSON Web Token (JWT) clock-injection tests discussed in Section 6.4, which were silently failing earlier and are now genuinely passing.

Table 6.1: Static verification commands and results.

Component	Command	Result
Go modules: control-plane, validator, gitops, deploy-engine	<code>go build ./... && go vet ./... && go test ./...</code>	Zero errors, zero vet findings, all tests pass
Python: causal-inference; validator Hypothesis sidecar [9]	<code>pytest</code>	38 tests passing; sidecar suite passing
apps/web — Next.js	<code>pnpm typecheck && pnpm lint && pnpm build</code>	All three gates pass

6.2 Live End-to-End Recovery Runs

Static verification establishes per-service soundness, not that the nine agents, the Temporal workflow [2], the validator sandbox, the approval gate, and the live console actually cooperate. Two recovery runs were therefore carried out against the real system: genuine PostgreSQL with Row-Level Security switched on, genuine Temporal workflows, a genuine Neo4j graph, and a real local LLM (Ollama [20] serving `qwen2.5-coder:14b` and `gpt-oss:20b`) generating the actual plans, diffs, and tests. Nothing was mocked or replayed.

6.2.1 Run 1: Approval-Gate Timeout — the Safety Mechanism Under Real Timing Pressure

A fault fixture was injected, Sentinel detected it, and the pipeline ran to completion: the Architect produced a plan, the Backend agent a unified-diff patch, the Quality Assurance (QA) agent Hypothesis property tests, and the Validator executed the patch against them inside its locked-down container sandbox, with every Layer 1 and 2 step turning up green. At the human approval gate, however, no decision arrived within the 120-second window, so the workflow resolved to a rejected terminal state.

This is a *positive* result: the central safety invariant is that no patch deploys without validation and approval, and that the system fails closed on any ambiguity, where an absent decision counts as refusal, never as consent. Rather than ship an unreviewed change, the system discarded a genuinely generated patch and durably recorded the timeout-rejected outcome. It is reported deliberately: a rejection is the stronger evidence that the gate is real.

6.2.2 Run 2: Approved and Succeeded in 1 Minute 40 Seconds

In the second run, the same detection path was followed, but approval was granted within the window through the console’s decision dialog. Classified as Medium severity, the patch passed sandbox validation, and the pipeline proceeded through deploy to a *Succeeded* state, 1 minute 40 seconds end-to-end. Two properties set this run apart from a staged demonstration: the timeline records genuine per-agent token and cost figures from the live Ollama provider, and the raw JSON of the Backend agent’s activity event contains the actual unified diff written by

qwen2.5-coder:14b (provider="ollama" appears in its metadata): the deployed patch is the very patch the model wrote, not a canned fixture.

6.3 Live Deploy-Engine Verification

The deploy-engine was run live against the public repository `heroku/node-js-getting-started`, chosen because it contains no `Dockerfile`, the engine's hardest path. It cloned the repository, detected the stack from marker files (`detected_stack: "node"`), generated a working `Dockerfile` from scratch (`dockerfile_source: "generated"`), built the image, and ran it under resource limits. Liveness was then proved directly: an HTTP GET on the preview URL returned the app's real rendered HTML, and the stop endpoint removed the container, three HTTP calls against a public repository, so the run is reproducible. Docker was unavailable on the development machine, so Podman [11] served as runtime through the same Docker-compatible API [10]. The clone was unauthenticated, so the GitHub App installation-token path went unexercised, a point noted among the limitations.

6.4 Defects Found and Fixed During This Engineering Cycle

Verification surfaced two genuine defects (Table 6.2). The JWT defect was *silent*, failing without blocking anything visible, and it was caught only because the full `go test ./...` output was read rather than skimmed. The migration defect, by contrast, manifests only at apply time.

Table 6.2: Defects discovered by verification.

Defect	Root Cause	Fix	Outcome
13 JWT tests silently failing	exp claims checked against the wall clock, not an injected test clock	Inject clock via <code>jwt.WithTimeFunc</code> (one line)	All 13 pass deterministically; production behaviour unchanged
Migration failed to apply	Reserved PostgreSQL keyword <code>current_role</code> used unquoted as a column name	Column renamed to <code>existing_role</code>	Applies cleanly; Go suites remain green

One further bug, pre-existing and out of scope, was deliberately left unfixed: the workspace-onboarding poller receives a 404 from `/v1/workspaces/{id}/events`, though the workspace provisions server-side. This is recorded among the limitations.

6.5 Testing Summary

Static verification came back 100% clean across every service, and on top of it sit three live runs: two nine-agent recovery executions exercising both approval-gate branches, plus one deploy-engine run, none simulated, every LLM step served by a real local model. Equally important is what was *not* tested: no formal load or performance testing was carried out, no throughput or latency under multi-tenant traffic is claimed, and the 1-minute-40-second figure reflects a single observed run, not a statistical claim. Load testing and a larger sample of recovery runs, sufficient to support meaningful Mean Time To Recovery (MTTR) distributions, remain future work (Chapter 7).

Chapter 7: Conclusion and Future Work

7.1 Overview

This project set out to close a loop that existing Artificial Intelligence (AI) developer tools leave open: from a detected software failure to a deployed, verified fix, with a human retained as a single decision point rather than the pipeline itself. Nexis, the platform delivered here, realises that goal as a working system, not a design proposal.

The central deliverable is the nine-agent closed-loop self-healing pipeline, durably orchestrated by Temporal [2]. A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) carries out the engineering work, while the four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detects faults, localises root causes over the Neo4j dependency graph [3], sandbox-validates every candidate patch, and routes it through a severity-classified Approval Gate offering Approve, Reject, and Modify. Chapter 6 exercised this pipeline live in two end-to-end recovery runs against a real local language model [20]. In one run, the patch was timeout-rejected exactly as the safety design intends, its 120-second approval window having elapsed; in the other, a detected fault travelled through planning, patch generation, property-based test validation, and human approval to a Succeeded deployment in roughly one minute and forty seconds.

The second major deliverable is a from-scratch, Docker-compatible preview-deploy engine [10]: it detects a repository's stack, generates a missing Dockerfile, builds and serves the container under resource limits, and returns a live preview URL. Its build and health-check failures feed back into the same self-healing loop, and the engine was proven live against a public repository that had no Dockerfile. Around both sits the complete multi-tenant platform: SSO/MFA authentication [12], row-level RBAC [7], Stripe billing [13], integrations with GitHub, Sentry, Datadog, PagerDuty, and Slack [14, 16, 15, 17], and OpenTelemetry observability [5] backed by a hash-chained audit log, all of it passing full build, lint, and test suites.

7.2 Limitations of the Project

Academic integrity demands that the boundary between what was demonstrated and what was not be stated in plain terms, not softened by hedging. As such, every limitation below is real, known, and deliberate; none of it is dressed up in gentler wording.

- **No production credentials were exercised.** The evaluation environment used for this report held no real GitHub App or OpenAI production credentials. Live evidence was produced instead with local Ollama models [20] and an unauthenticated clone of a public repository, so a fully credentialed production path, ending in a real, merged GitHub pull request, was never exercised end-to-end.
- **The platform is not yet live on its domain.** The `nexis.sh` domain is registered, but the platform has not yet been deployed to it, and every piece of evidence in this report comes from local, native execution. Docker itself was unavailable in the build environment, so Podman [11] was substituted as a runtime compatible with the Docker Application Programming Interface (API) and shown to behave identically.
- **Fine-tuning is a manual step by design.** The Reinforcement Learning from Human Feedback (RLHF) data-export pipeline is real and has been tested: every terminal approval decision, including engineer-edited diffs, is persisted and exportable as JSON Lines (JSONL). Submitting a fine-tuning job against that export, however, is left as a deliberate manual step, since it involves real money and a real external API call rather than an automated one.
- **Only one of twenty-seven demo scenarios is fully wired.** The Live-Demo console lists twenty-seven curated fault scenarios, of which exactly one is wired end-to-end to a real fixture and classification path today. The remaining twenty-six are complete on the User Interface (UI) side but still backend-pending.
- **The causal ranking is a proxy, not a fitted causal model.** The causal-inference sidecar ranks root-cause candidates through an honest statistical and graph-structural scoring method, backed by a transparent, per-component confidence breakdown. Explicitly, it is not a fitted DoWhy structural causal model [4], since no interventional production data exists against which such a model could be fitted, and claiming otherwise would overstate what has been built.

- **A known, unrelated bug was left unfixed.** A pre-existing bug in the workspace-onboarding progress poller, a 404 returned from `/v1/workspaces/{id}/events`, turned up during evaluation and was intentionally left unfixed as out of scope. Regardless, the workspace itself provisions successfully on the server side.
- **No formal load testing was performed.** As Chapter 6 notes, no formal concurrent-load performance benchmark was run, and the latencies reported there come from single-run live executions rather than from a controlled load harness.

7.3 Future Work

Each item below is grounded in the existing architecture: it names a concrete next step rather than a generic aspiration.

- **Credentialed production deployment** to the registered `nexis.sh` domain, with a real GitHub App and a paid OpenAI account, is the only way to exercise authenticated cloning, real pull-request creation, and merge-triggered redeploy end-to-end.
- **Complete the scenario library.** Wiring the remaining twenty-six Live-Demo cards to real fixtures would extend fault-injection coverage across all five scenario categories.
- **Per-service Sentinel baselines.** Splitting the per-organisation EWMA baseline down to the level of each service would stop one noisy service from masking a quiet one.
- **A fitted causal model.** Once interventional production data exists, a real DoWhy model [4] can be fitted and evaluated against the deliberately replaceable graph-evidence proxy.
- **Execute a fine-tuning run** on the RLHF export to measure whether a tuned model lowers the rejection and modification rates seen at the gate.
- **Real preview subdomains**, served through a Traefik-style reverse proxy instead of `localhost` ports, would let previews be shared beyond one machine.
- **Formal load benchmarking** to characterise throughput, queueing behaviour under simultaneous incidents, and Temporal worker saturation.
- **Human-subjects usability study.** The NASA-TLX instrument [18] is already wired into the post-incident flow; only recruitment and analysis remain.

7.4 Conclusion

The central claim is deliberately narrow, yet deliberately strong. A software fault can travel from automatic detection, through multi-agent diagnosis, patch generation, and sandboxed validation, past a single human approval, to a deployed fix, inside one continuous, durable pipeline. This is demonstrated rather than asserted: once, the loop refused to proceed when its approval gate went unanswered; once, it carried a real fault to an approved, deployed resolution in under two minutes. Equally deliberate is what the project does not claim. Section 7.2 draws the evidence boundary exactly where the evidence stops, and within it, Nexis thus shows that human-gated, closed-loop fault recovery is buildable today.

References

1. GitHub Copilot. AI pair-programming assistant for code completion and suggestion. Available at: <https://github.com/features/copilot> (Accessed: 11 August, 2026).
2. Temporal. Durable workflow orchestration engine for fault-tolerant distributed applications. Available at: <https://temporal.io/> (Accessed: 11 August, 2026).
3. Neo4j. Native graph database for storing and querying highly connected data. Available at: <https://neo4j.com/> (Accessed: 11 August, 2026).
4. DoWhy. Causal inference library from Microsoft Research / PyWhy for modelling and estimating causal effects. Available at: <https://github.com/py-why/dowhy> (Accessed: 11 August, 2026).
5. OpenTelemetry. Open-source observability framework for traces, metrics, and logs. Available at: <https://opentelemetry.io/> (Accessed: 11 August, 2026).
6. Argo CD. Declarative GitOps continuous delivery tool for Kubernetes. Available at: <https://argo-cd.readthedocs.io/> (Accessed: 11 August, 2026).
7. PostgreSQL Row Security Policies. Official documentation on row-level security for multi-tenant data isolation. Available at: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (Accessed: 11 August, 2026).
8. OpenLineage. Open standard for data lineage collection and analysis. Available at: <https://openlineage.io/> (Accessed: 11 August, 2026).
9. Hypothesis. Property-based testing library for Python. Available at: <https://hypothesis.readthedocs.io/> (Accessed: 11 August, 2026).
10. Docker. Container platform for building, shipping, and running applications. Available at: <https://www.docker.com/> (Accessed: 11 August, 2026).
11. Podman. Daemonless, Docker-API-compatible container engine. Available at: <https://podman.io/> (Accessed: 11 August, 2026).
12. WorkOS. Enterprise-ready authentication and single sign-on platform. Available at: <https://workos.com/> (Accessed: 11 August, 2026).
13. Stripe. Online payment processing and billing infrastructure. Available at: <https://stripe.com/> (Accessed: 11 August, 2026).
14. Sentry. Application error tracking and performance monitoring platform. Available at: <https://sentry.io/> (Accessed: 11 August, 2026).
15. PagerDuty. Incident response and on-call management platform. Available at: <https://www.pagerduty.com/> (Accessed: 11 August, 2026).
16. Datadog. Cloud-scale monitoring and observability service. Available at: <https://www.datadoghq.com/> (Accessed: 11 August, 2026).
17. Slack API. Platform APIs for building Slack integrations and interactive messages. Available at: <https://api.slack.com/> (Accessed: 11 August, 2026).
18. NASA Task Load Index (NASA-TLX). Subjective workload assessment instrument developed by NASA Ames Research Center. Available at: <https://humansystems.arc.nasa.gov/groups/tlx/> (Accessed: 11 August, 2026).
19. pgvector. Open-source vector similarity search extension for PostgreSQL. Available at: <https://github.com/pgvector/pgvector> (Accessed: 11 August, 2026).

20. Ollama. Tool for running large language models locally. Available at: <https://ollama.com/> (Accessed: 11 August, 2026).